

Sentiment Classification of IMDB Movie Reviews Using Machine Learning

Jason Yu

Department of Computer Science
Georgia State University
Atlanta, United States
jyu37@student.gsu.edu

Abstract—This project was designed to use machine learning as a sentiment classifier on IMDB movie reviews. We used a dataset containing 50,000 reviews that is balanced and labeled. The sentiment classifier will be used to classify text as positive or negative. The dataset was preprocessed with techniques such as HTML tag stripping, stop word removal, and capitalization/punctuation removal. The cleaned data then went through TF-IDF vectorization for PCA and for the models to train on. We trained and evaluated multiple classifiers to determine which would work best for our problem task, such as Naive Bayes, Logistic Regression, a Support Vector Machine(SVM), Random Forest, and XGBoost. Logistic regression and SVM performed the best with accuracy scores of 0.89 and 0.90, respectively. We explored interpretability by visualizing word weights and PCA. We also deployed our final model as a web application using Streamlit. This project demonstrates the effectiveness of traditional machine learning techniques.

I. INTRODUCTION

A. Project Overview

The goal of this project is to train a model that can classify a movie review as positive or negative. This project also aims to experiment with different machine learning algorithms to determine which can provide the best result for this task. The approach to this project is to use Term Frequency-Inverse Document Frequency (TF-IDF) first to vectorize the dataset. Then we can use the resulting matrix as the input to train our dataset on.

B. Dataset

For this project, we are using a dataset of 50,000 IMDB reviews from Kaggle. These reviews are balanced with 25k positive and 25k negative reviews. The dataset is also properly labeled with its sentiment. The resulting dataset is a two column matrix with the first containing the review and the second containing the sentiment. This makes it an ideal dataset to work with for a machine learning project

II. LITERATURE REVIEW

Sentiment analysis is a type of Natural Language Processing (NLP) task. It mainly involves the ability to identify and classify the tone and opinion expressed in text. Its applications can range from product reviews to social media monitoring. While early approaches involved a lexicon-based and rule-driven based system, modern sentiment classification relies heavily

on machine learning algorithms trained on labeled data. For English datasets, such as the IMDB movie dataset were using, this shift has led to a pipeline of text preprocessing, feature extraction (TF-IDF), and the use of a classification model.

A study done by Chongqing University conducted tests to see which machine learning model works best for sentiment analysis [1]. They claim a Support Vector Machine is stated to be the most famous algorithm for it, but it requires a long time to train large datasets. Other models that can be considered for our project could be a Naive Bayes, a Logistic Regression, or a decision tree model for their simplicity.

III. TEXT PREPROCESSING AND TF-IDF

For NLP, an important phase of the pipeline is to preprocess the code. This is to ensure the proper word frequencies are shown in the feature extraction. It's also important that we show the dataset to the model in a way that it can understand. We do this by using a TF-IDF vectorizer on the data to convert text data into a numerical format.

A. Cleaning the Text

We must simplify the dataset by removing any capitalization or punctuation. For our case, if we run our dataset through a TF-IDF vectorizer, it will treat "great," "Great," and "great!" as different tokens. This can affect the models we train, for example, in logistic regression, the weights of the words will be split between two different tokens. Another issue to address is the fact that the dataset also contained HTML tags. These are used in the formatting of webpages, but they show up frequently in our dataset, especially the line break tag(br). We can further clean up the dataset by using the Python library, BeautifulSoup, to remove all HTML tags from our dataset.

B. Term Frequency-Inverse Document Frequency

TF-IDF is a tool that can measure word frequency [3], but it measures how common a word is alongside how rare it is. It does this by combining two components: Term Frequency and Inverse Document Frequency. Term frequency (TF) measures how often a word appears in a document, a document in our case is an individual review. A higher appearance usually

means that the word has some importance to the context of the document.

$$TF = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d} \quad (1)$$

TF is weird because it doesn't consider the word's importance outside of the document. This is why we also consider Inverse Document Frequency. IDF measures how rare a word is across all documents in the dataset. This is because rarer words usually have more meaning or are more specific to the document.

$$IDF = \log \frac{\text{Total number of documents in dataset } D}{\text{Number of documents containing term } t} \quad (2)$$

IDF alone is also weird in the sense that it might give a high value to a random, insignificant word. We solve them by just multiplying TF and IDF together to get an TF-IDF value. We can use a TF-IDF vectorizer to convert a list of documents into a matrix, with each row pertaining to a document, each column pertaining to a vocabulary built with all the words in the dataset, and each entry being the TF-IDF score of the word in the document.

IV. EXPLORATORY DATA ANALYSIS (EDA)

EDA is a way for us to analyze and visualize a dataset [2]. It's an important step in machine learning as it allows us to determine patterns that might be useful in how we select which model to use and how to optimize them.

A. Checking to See if the Dataset is Balanced

The first EDA that was conducted was checking to see if the dataset is balanced. Although we knew that it was balanced already, it doesn't hurt to be able to double-check the dataset by writing a program to count how many reviews of each label appear

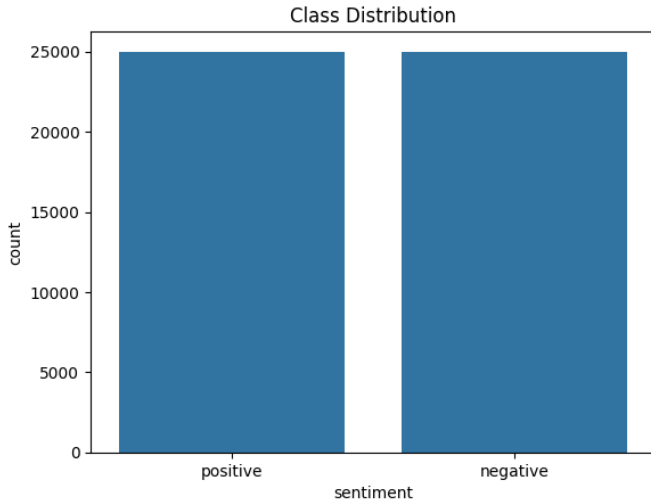


Fig. 1: Balanced Dataset.

B. Finding Out the Top Words In Reviews

This EDA is designed for us to count which words appear the most often in both the positive and negative reviews. Since we want to extract features using TF-IDF, this will help give us a sense of which words might be favored by the vectorizer. The first problem that came up when running this EDA was the fact that the dataset hadn't been processed yet. This meant that when we tried to count the words, the top word for both positive and negative was "br". At first br seems like a strange word to be contained in a movie review until we reviewed the dataset. The "br" word stemmed from the HTML break line tag. This posed a huge issue since the tag appeared almost 100,000 times in the positive reviews alone. We can fix this

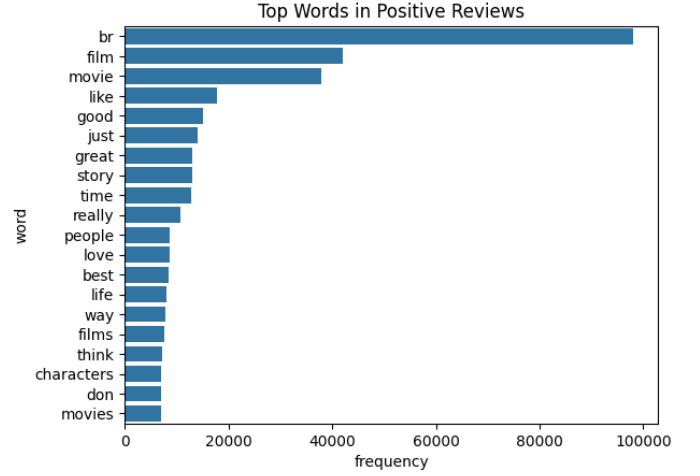


Fig. 2: Top words before text cleaning.

by removing the HTML tags in the text preprocessing script. This ensures the only thing in the reviews is the actual context of the review. Now we can compare the words in the positive and negative reviews.

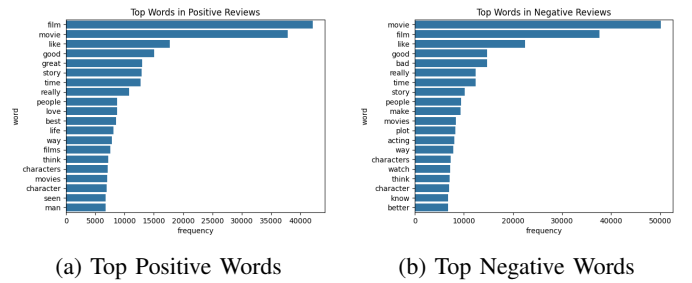


Fig. 3: Top words of positive and negative side by side

One thing to keep in mind with this EDA is that the words "movie," "film," and "like" are the top three words of both negative and positive. However, after, we can see that the top words are more correlated with the label, such as "good" and "great", labeled as positive.

C. Principal Component Analysis

After TF-IDF, the resulting matrix has a size of Number of Documents in Dataset x Number of Unique Words in Dataset.

This results in a very high-dimensional matrix, which can be quite challenging to visualize. We can get around this by visualizing with Principal Component Analysis (PCA). PCA is a dimensionality reduction technique that transforms the principal components into a new coordinate system. PCA is

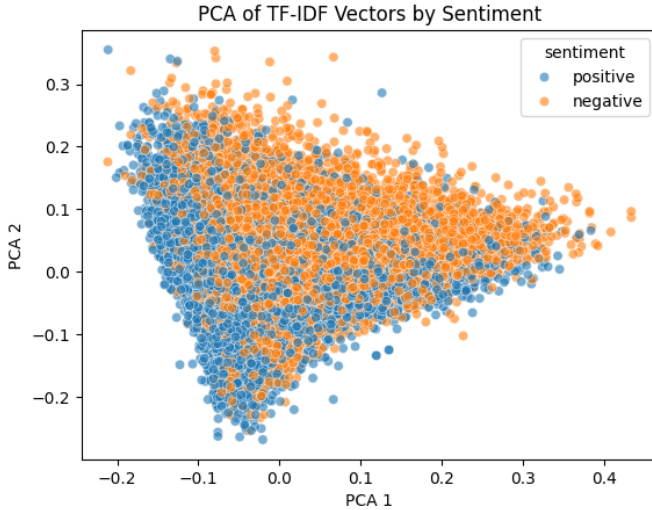


Fig. 4: PCA of the TF-IDF matrix.

interesting because the closer a point is to another on the plot, indicates how similar they are. With the PCA we can see that while there is no clear boundary between the sentiments, we can see that there is clustering between the two. The fact that there isn't a clear boundary isn't too worrisome because we are compressing a very high-dimensional matrix into two dimensions, so a lot of information is bound to be lost.

V. METHODOLOGIES AND ALGORITHMS

For this project, there were a couple of different models that we trained. We have a Naive Bayes, Logistic Regression, 4 SVMs (Linear, Radial Basis Function, Poly, and Sigmoid), Random Forest, and XGBoost. For machine learning on a TF-IDF matrix, each feature is a word from the vocabulary of the dataset.

- **Naive Bayes(NB):** NB is a classification algorithm that is based on Bayes' Theorem that also assumes that each feature is unrelated [4]
- **Logistic Regression (LR):** LR uses a sigmoid function to predict inputs into a probability value between 0 and 1. [5] LR assigns each feature a weight based on its sentiment, which is nice because we can then compare the weights of the words individually.
- **SVM:** A type of model that classifies data by finding a hyperplane that separates the data into its two labels. You can also use kernel functions to transform the data, such as RBF, polynomial, and sigmoid.
- **Random Forest (RF):** A decision tree algorithm that randomly contains a bunch of decision trees trained on a random subset of text. Whatever the majority of the

trees agree on is how the input is classified. Decision tree algorithms, in general, are not great with text data.

- **XGBoost:** Another type of decision tree algorithm that relies on training trees that correct the mistakes of the previous ones. Similarly with RF, they are not the greatest with text.

A. Optimizing

With certain simple models, we also used Grid Search to optimize the models. How that works is that a list of different parameters is imputed it will provide the model that has the best value of a desired trait. In our case, we wanted the parameter that provided the best accuracy. We conducted a grid search optimization on NB and LR since training the SVM models took too long.

VI. DATA AND RESULTS

A. Metrics Used

When evaluating models, we can use precision, recall, F1-score, support, and accuracy to compare different models

- **Precision:** Out of all the predictions made for the class, how many were correct? High precision means that there are fewer false positives
- **Recall:** Out of all the actually labeled reviews in a class, what percent did the model correctly identify? High recall means that there are fewer false negatives
- **F1-Score:** Combination of precision and recall
- **Support:** Number of samples in the class in the test set
- **Accuracy:** Total Correct Predictions / Total Samples

Precision vs Recall can be confusing because it sounds like they're measuring the same thing, but an analogy to differentiate them is if you are designing an email spam filter. Precision is, out of all the emails marked as spam, how many were actually spam. Recall is, out of all the actual spam emails, how many did the filter actually catch.

B. Data Table

Below is a table that compares the precision, recall, F1-Score, and Accuracy of all the models we trained. For each model it compares the scores of both classes, 0 and 1, where 0 represents the negative class and 1 represents the positive class. Since all the models were trained and tested on the same 80/20 data split, all the models have a support of 0: 5008 and 1: 4992.

C. Confusion Matrices

A confusion matrix is a heatmap that is able to show information on how your model performed beyond just an accuracy number. It shows the actual number of predictions for each class and how many align with its true label. It then uses a color gradient to compare the size of the numbers with other squares. In general, confusion matrices are more interesting when comparing datasets of a large number of classes, but it is still interesting to see how they appear for binary classification. Since the Logistic Regression, SVM (RBF), and SVM (Sigmoid) had the highest accuracy with

TABLE I: Model Results

Model	Label	Precision	Recall	F1 Score	Accuracy
NB	0	0.86	0.84	0.85	0.85
	1	0.85	0.86	0.85	
LR	0	0.90	0.87	0.89	0.89
	1	0.87	0.90	0.89	
SVM (Linear)	0	0.89	0.87	0.88	0.88
	1	0.87	0.89	0.88	
SVM (RBF)	0	0.90	0.88	0.89	0.90
	1	0.89	0.91	0.90	
SVM (Poly)	0	0.90	0.84	0.87	0.87
	1	0.85	0.91	0.88	
SVM (Sigmoid)	0	0.89	0.88	0.88	0.89
	1	0.88	0.89	0.89	
RF	0	0.84	0.85	0.85	0.85
	1	0.85	0.84	0.84	
XGB	0	0.88	0.83	0.85	0.86
	1	0.84	0.88	0.86	

0.89, 0.90, and 0.89, respectively. Here are the confusion matrices for those three.

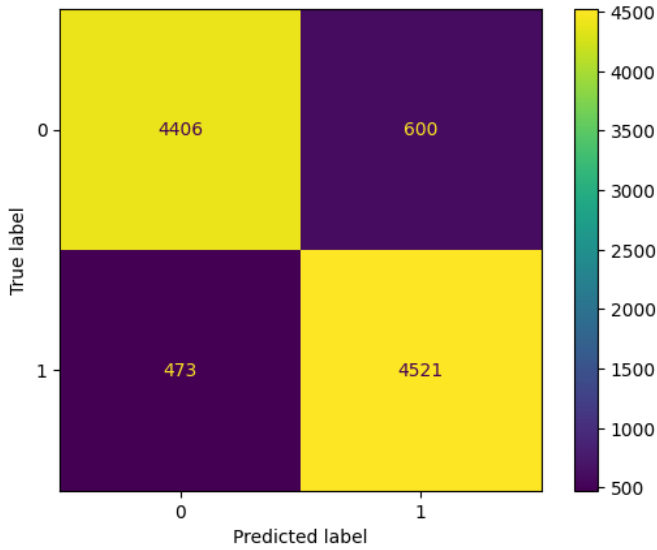


Fig. 5: Logistic Regression Matrix

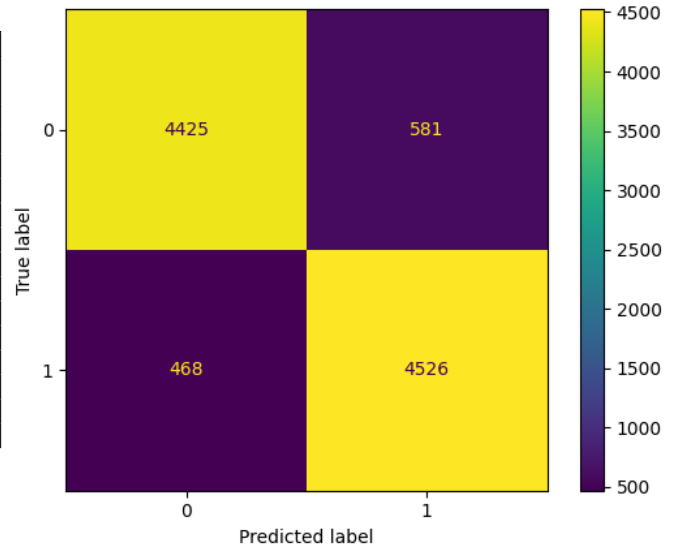


Fig. 6: SVM (RBF)

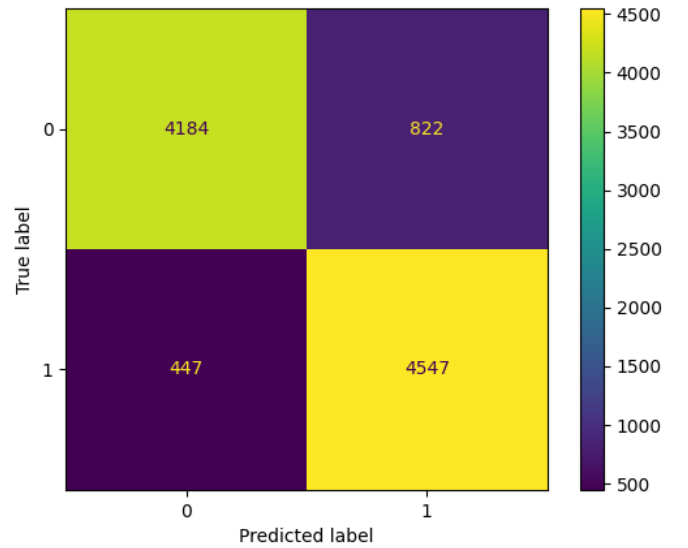


Fig. 7: SVM (Sigmoid)

D. Evaluating Results of Metrics and Confusion Matrices

If we compare purely off accuracy, LR, SVM (RBF), and SVM (Sigmoid) all scored relatively well and similarly. One thing to consider when at SVM (Sigmoid), however, is that its recall score for label 0 is pretty low in comparison to the others. We can all see this in the confusion matrix, where there is a noticeable difference in the colors for label 0. Furthermore, if we compare the scores between LR and SVM (RBF), RBF matches or exceeds the scores of LR. With this analysis, SVM (RBF) is probably the best model to move forward with.

VII. RESULTS

A. Demo Application

Since we finalized on SVM (RBF), we can then save the model to be used for further testing. In the case of this project,

we designed a web application using Streamlit that allows a user to input a movie review that the model will then try to classify. We can use this to determine what the model struggles with.

B. Visualizations

For Logistic Regression, in principle, it works by assigning weights to each feature, in our case, words. We can visualize this by putting it into a bar plot. This allows us to see which words are correlated with positive and negative labels.

C. Problems with the Project

While having an accuracy score of 0.90 isn't too bad, there is still room for improvement. When playing around with the model application, we noticed some flaws with the model. The

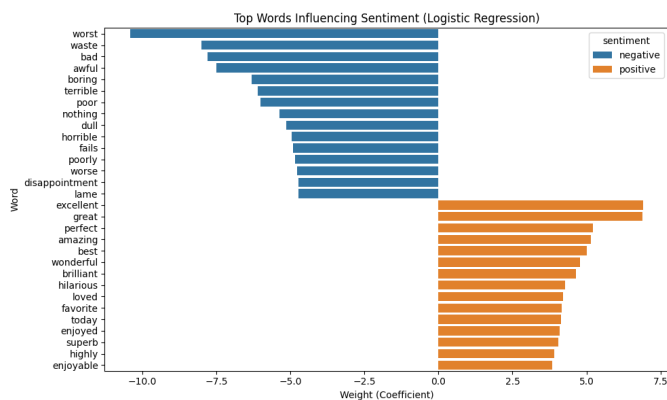


Fig. 8: LR Word Weights

first thing we noticed is the fact that the word "not" is included in NLTK's stop word list. This resulted in a negative sentence like "this movie is not great" turning into a "movie great" after text cleaning. If you refer back to our logistic regression plot, we can see that great has a positive connotation. When using our Logistic Regression model, the previous negative sentence is labeled positive with a 0.99 confidence score. To combat this, our first idea was to just include "not" back into the vocabulary of the model. However, this ended up not changing much. This is because it does not consider that "not" may not have any sentiment alone; it is a negation word. This means that it flips the sentiment of the word it precedes, so if we want to find a solution, we'll have to target negation words in general.

Another issue that was noticed when using the model was that the tense of a word might also play a role in affecting the connotation of certain words. For example, "hate" is normally a negative word. However, in the eyes of the model, it has a positive connotation. When doing some more testing, it was discovered that the word "hated" was properly labeled as negative. Going back to our TF-IDF vectorizer, "hate" and "hated" are treated as different word tokens. This means that one of them might not gain as much weight as the other during training, even though the only difference in English is the tense. To target this, a way to equate a word with its different tenses must be found to remove this issue.

When presenting in class, a problem that arose was that of typos. If a keyword is misspelled in the input, it can drastically affect the output. This is because since typos are most likely very unique, it's unlikely that they would show up in the training data and in the vocabulary of the model that was trained.

VIII. CONCLUSION

In conclusion, we can say that with our current setup, a Radial Basis Function Support Vector Machine works the best with what we're given. We also learned that it is important to make sure your text dataset is properly processed before any machine learning training can be conducted on it. If this project were to be extended, we would look into finding

solutions to the problems with the text dataset mentioned above. We would also look into training some other models, such as a shallow neural network. Another path that would be interesting to pursue would be how well the model can handle sarcasm. However, with what we have now, we are happy with how the current model performs in the time frame given. This project has given much insight on the pipeline of how machine learning for NLP works.

REFERENCES

- [1] Yangying Mao, Qun Liu, Zhang Yu, "Sentiment analysis methods, applications, and challenges: A systematic literature review," Journal of King Saud University - Computer and Information Sciences, Volume 36, Issue 4, 2024. <https://doi.org/10.1016/j.jksuci.2024.102048>
- [2] <https://www.ibm.com/think/topics/exploratory-data-analysis>
- [3] <https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/>
- [4] <https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/>
- [5] <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>